

Safety Goggles Detection Using Edge AI Acceleration

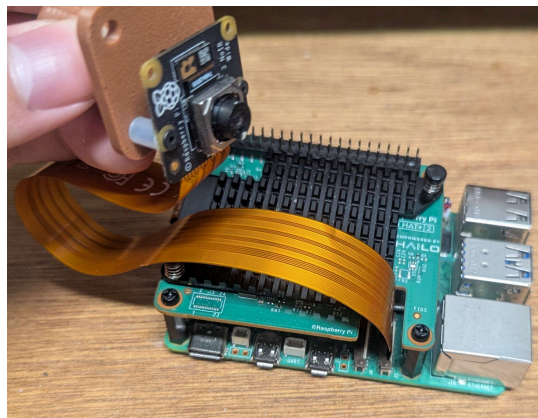
By: Wenkai (Kevin) Chen

ABSTRACT

This project presents a real-time Safety Goggles Detection System designed for the Georgia Tech Invention Studio, with the goal of improving PPE compliance in machine shop environments. Building upon previous CPU-based implementations, this work integrates a Raspberry Pi 5 with a Hailo-10H AI accelerator (AI Hat + 2) to enable efficient, low-latency edge inference. A fine-tuned YOLOv8s model is used to detect PPE-related features based on a relabeled dataset.

The system follows an end-to-end deployment workflow, from dataset relabeling and model training to hardware-specific compilation and real-time inference. Due to the complexity of distinguishing between regular prescription glasses and safety goggles, the classification task was simplified to focus on more robust and generalizable categories. A key challenge in this process is converting trained models into formats compatible with the Hailo accelerator, as mismatches in Ultralytics ONNX export versions can lead to incompatibility during compilation.

While a fully complete deployment was not achieved within the scope of VIP, a near-complete system has been developed, including model training, hardware integration, and a functional inference pipeline. As a single-member subteam effort, this project demonstrates both the feasibility and the challenges of deploying optimized computer vision models on edge hardware in real-world conditions.



INTRODUCTION

The Georgia Tech Invention Studio is a student-run makerspace that provides access to a wide range of fabrication tools, including machine shops, laser cutters, and electronics workstations. While this open environment promotes creativity and hands-on learning, it also introduces challenges in maintaining consistent safety practices. Eye protection, in particular, is critical when operating machinery, yet ensuring compliance remains difficult due to the lack of continuous supervision and high variability in user behavior.

Previous work has explored the use of computer vision systems to automate PPE compliance by detecting safety goggles at entry points. These systems typically rely on embedded platforms such as the Raspberry Pi 5 running lightweight object detection models like YOLOv8. While effective in controlled scenarios, such approaches are often limited by computational constraints, resulting in reduced inference speed and inconsistent real-time performance.

This project extends prior work by introducing dedicated edge AI acceleration to address these limitations. By integrating a Hailo-10H accelerator with the Raspberry Pi 5, the system is capable of running optimized neural network inference directly on-device, significantly improving performance while maintaining low power consumption. A custom dataset from the previous semester was fully relabeled to better reflect real-world conditions, and a YOLOv8s model was fine-tuned to classify simplified PPE-related categories.

Beyond dataset preparation and model training, a major focus of this work lies in the complete deployment workflow required for edge inference. This includes exporting trained models to ONNX format, compiling them into hardware-specific HEF binaries using the Hailo toolchain, and integrating them into a real-time detection pipeline. Throughout this process, challenges such as ONNX format inconsistencies and toolchain compatibility emerged as key obstacles, requiring iterative debugging and validation.

The resulting system performs real-time detection and provides immediate visual feedback through an LED indicator, serving as a proactive safety reminder for users entering the workspace. As an independently developed project, this work encompasses all stages from data processing to hardware deployment, demonstrating a practical edge AI solution for improving safety compliance in makerspaces.

MODEL SETUP

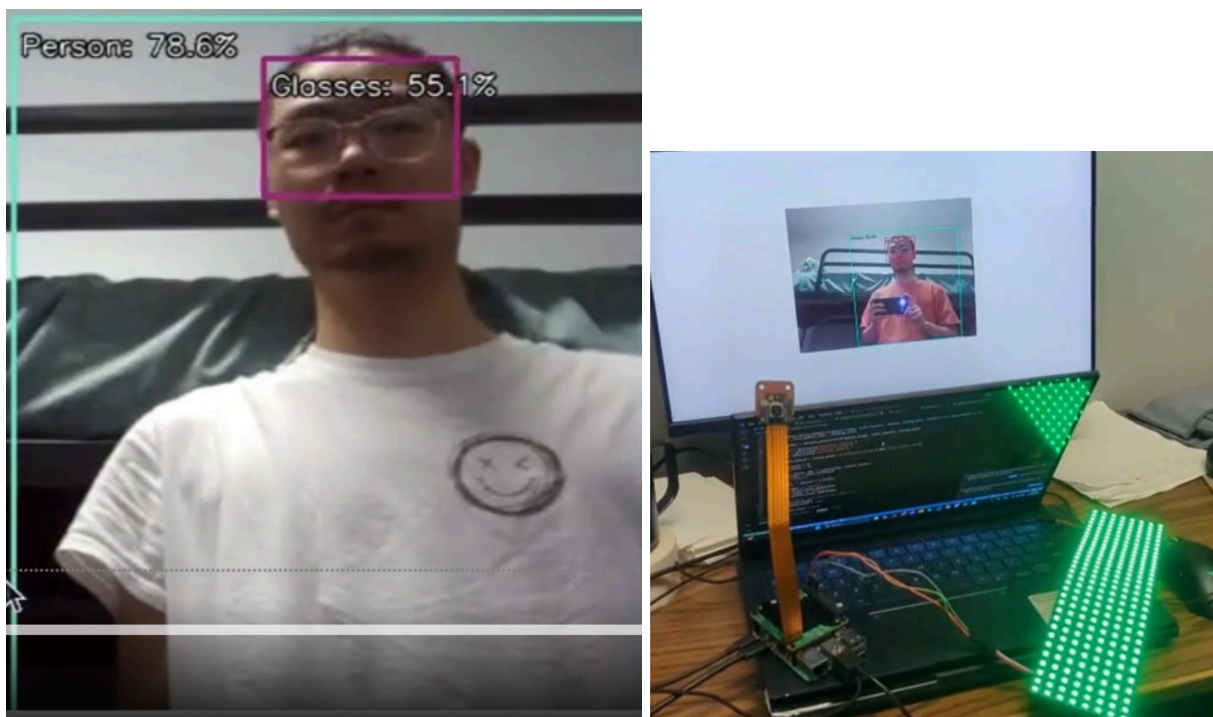
Hardware Setup

The system is built on a Raspberry Pi 5 paired with a Hailo-10H AI accelerator (AI Hat + 2), enabling hardware-accelerated neural network inference. A camera module provides continuous video input for detection. An LED strip is connected through GPIO to deliver immediate visual feedback, where non-compliance triggers a warning signal.

Software Stack

The model is developed using the Ultralytics YOLOv8 framework. A dataset from the previous semester was fully relabeled and restructured, with configuration defined through a dataset YAML file. Due to the difficulty of distinguishing between safety goggles and regular eyeglasses, the dataset was simplified to include more generalizable classes, such as person and glasses.

The YOLOv8s model was fine-tuned on this relabeled dataset to improve robustness under real-world conditions. The trained model is exported to ONNX format and compiled into a HEF (Hailo Executable Format) using the Hailo Model Zoo and Dataflow Compiler. The final inference pipeline is implemented using the Hailo Apps environment, where a modified standalone detection script runs the custom HEF model with corresponding label configurations.



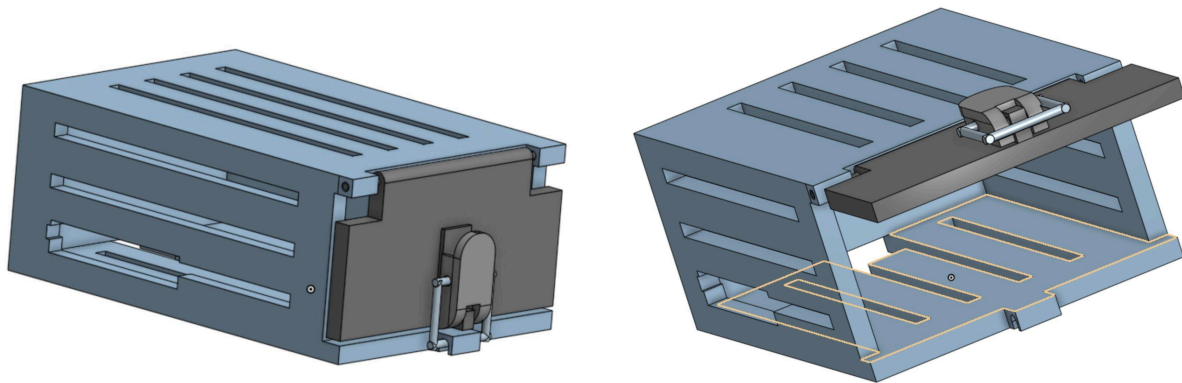
IMPLEMENTATION

Building on the hardware and software setup, the implementation follows a complete edge AI deployment workflow. The YOLOv8s model was fine-tuned using the relabeled dataset and corresponding YAML configuration, improving model stability across variations in lighting, distance, and user appearance.

A critical step in deployment is converting the trained model into a hardware-compatible format. The model is first exported to ONNX and then compiled into a HEF file for execution on the Hailo accelerator. This compiled model is integrated into the Raspberry Pi system using the Hailo Apps framework, enabling real-time inference on live camera input.

The detection output is linked to an LED feedback system. When the system detects missing or improper PPE-related features, a visual warning is triggered. The system operates continuously and is designed for plug-and-play deployment at machine shop entry points.

In addition to the software pipeline, a custom 3D-printed enclosure was designed to support the updated hardware configuration. Due to the addition of the AI Hat + 2, the original mounting design for the Raspberry Pi was no longer compatible, as the accelerator occupied the available mounting holes. To address this, a new enclosure was designed and fabricated to securely mount both the Raspberry Pi and the accelerator while maintaining accessibility for power, camera, and GPIO connections. This redesign improves mechanical stability and enhances the overall deployability of the system.



Deployment

The deployment process started with preparing the custom dataset in Roboflow. I uploaded the collected person with safety-goggles images into Roboflow and manually checked the annotations to make sure each image was labeled correctly. Since distinguishing between regular prescription glasses and actual safety goggles was difficult in real-world conditions, I simplified the dataset into more reliable classes, such as Person and Glasses. After the relabeling process was complete, I generated a YOLOv8-compatible dataset export from Roboflow. This export included the training, validation, and testing image folders, along with a data.yaml file that defined the class names and dataset paths.

After preparing the dataset, I fine-tuned a YOLOv8s model using Ultralytics (version: 8.3.252). The training process used the exported data.yaml file from Roboflow so that the model could learn from the custom PPE dataset. A typical training command was:

```
!yolo task=detect mode=train model=yolov8s.pt data=/path/to/data.yaml (need modification)
epochs=100 batch=16 imgsz=640 plots=True
```

Once training was complete, the best-performing model weight file, usually named best.pt, was selected from the training results folder. This model was then exported into ONNX format so it could be used with the Hailo compilation toolchain:


```
lyolo export model=/path/to/best.pt format=onnx imgsz=640
```

The ONNX file was then converted into a Hailo Executable Format file, or .hef, which is the format required to run inference on the Hailo-10H accelerator. This step used the Hailo Model Zoo and Hailo compiler. Since the Hailo compiler only supports a Linux environment, this process was performed in a Linux-based system (such as Ubuntu or WSL). The compilation command included the ONNX checkpoint, the target hardware architecture, the calibration image folder, and the number of classes:

```
hailomz compile yolov8s \
```

```
--ckpt=/path/to/best.onnx (need modification) \
```

```
--hw-arch hailo10h \
```

```
--calib-path /path/to/train/images \
```

```
--classes 2 \
```

```
--performance
```

The calibration path is important because the Hailo compiler uses sample training images to optimize the model for the accelerator. One of the main difficulties during this step was that different Ultralytics versions can export slightly different ONNX structures. Some exported ONNX files were not compatible with the Hailo Model Zoo pipeline, so I had to test different export versions and debug the model conversion process.

After the .hef file was generated, I moved the compiled model onto the Raspberry Pi 5. The Raspberry Pi was set up with the Hailo Apps environment, which provides example pipelines and Python scripts for running inference with the Hailo accelerator. I installed and activated the Hailo Apps virtual environment, then used the standalone object detection pipeline as the base for my custom goggles detection program.

The final custom detection file, goggle_detection, was placed inside the Hailo Apps object detection directory. This script loads the custom .hef model (named yolov8.hef in the github repo), connects to the Raspberry Pi camera input, runs real-time inference through the Hailo-10H accelerator, and applies the custom label configuration. The basic runtime flow is:

```
cd ~/hailo-apps
```

```
source setup_env.sh
```

```
cd PPE
```

Then

```
python goggle_detection.py \
```

--input rpi \

--hef-path yolov8s.hef \

--labels-json goggle_labels.txt

Very useful resource would be:

https://youtu.be/Dm37x7sObIc?si=BOwn_mEojsHp-_a2

<https://youtu.be/7pgSFgqo8gY?si=ZHDW4RB3QZFhH8MC>

The detection result is then connected to the LED feedback system. If a person is detected without the required PPE-related feature, the LED indicator turns red as a warning. If the system detects the expected safety-related class, the LED turns green. This creates a full edge-AI pipeline: camera input, Hailo-accelerated inference, decision logic, and real-time visual feedback.

PROBLEMS FACED

Several key challenges were encountered during the development of this system:

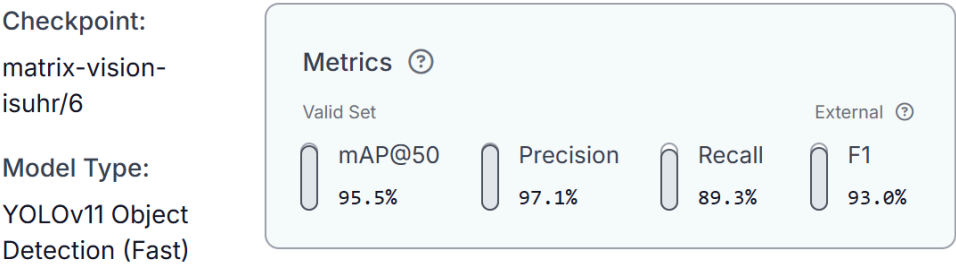
- **ONNX Export Incompatibility** — Different versions of the Ultralytics framework produce inconsistent ONNX outputs, some of which are incompatible with the Hailo Model Zoo compilation pipeline. This became the primary bottleneck in deployment.
- **Toolchain Integration Complexity** — Converting ONNX models into HEF format requires strict alignment between model structure, YAML configuration, and Hailo compiler expectations, making debugging difficult and time-consuming.
- **Edge Deployment Constraints** — While the Hailo accelerator improves inference performance, integrating it with the Raspberry Pi environment and ensuring correct runtime configuration required significant experimentation.
- **Dataset Limitations** — Simplifying the dataset to more general classes improves robustness but reduces classification granularity, limiting the system's ability to distinguish between safety goggles and regular eyeglasses.
- **Single-Person Development Scope** — As a one-person subteam, all aspects of the project—including data processing, training, hardware integration, and deployment—were handled independently, increasing development complexity and limiting overall project scope.
- **Limited Documentation and Resources** — There is a lack of comprehensive documentation for deploying custom computer vision models using the AI Hat + 2 and Hailo-10H accelerator, particularly when integrating with custom-trained YOLO models.

This required extensive independent experimentation, including adapting existing Hailo examples and troubleshooting undocumented toolchain behaviors.

RESULTS

A key objective of this project was to improve detection accuracy and robustness by refining the dataset and optimizing the deployment workflow. Compared to the previous implementation, which relied on a smaller and less consistent dataset, this work involved fully relabeling and restructuring the dataset to better reflect real-world conditions in the machine shop environment.

The updated model demonstrates strong performance across evaluation metrics. On the validation set, the model achieved:



These results indicate a substantial improvement in detection reliability. The high precision indicates fewer false positives, while the improved recall reflects better generalization across varied conditions such as lighting, distance, and user variability.

In comparison to the previous system, which achieved approximately 83% precision and 81% recall, this model shows clear improvements in both accuracy and consistency. This improvement is primarily attributed to the fully relabeled dataset, improved annotation quality, and better alignment between training data and deployment conditions.

In addition to accuracy improvements, the integration of the Hailo-10H accelerator enables real-time inference performance that was not achievable in the previous CPU-only implementation. While full system deployment was not completed, the inference pipeline demonstrates stable and efficient real-time behavior.

Overall, the system represents significant progress toward a functional minimum viable product, with core components successfully developed and validated.

CONCLUSION

This project presents a Safety Goggles Detection System that leverages edge AI acceleration to improve both accuracy and real-time performance in PPE compliance monitoring. By integrating a Raspberry Pi 5 with a Hailo-10H accelerator, the system addresses the limitations of previous CPU-based implementations and enables efficient on-device inference.

A major contribution of this work is the development of a complete end-to-end system, including dataset relabeling, YOLOv8s model fine-tuning, ONNX export, hardware-specific compilation, and edge deployment integration. Due to the complexity of distinguishing between regular eyeglasses and safety goggles, the classification problem was simplified to improve robustness and feasibility within the project scope.

Although a fully complete minimum viable product was not achieved, the project successfully implements the majority of the system components and demonstrates a working inference pipeline on edge hardware. The results highlight both the potential and the challenges of deploying computer vision systems in real-world environments.

As a single-member subteam effort, this project represents a substantial step toward a deployable solution. This work establishes a strong foundation for future development toward a fully integrated safety monitoring system in the Invention Studio.

Reference

Ai Hat + 2 documentation:

<https://www.raspberrypi.com/documentation/accessories/ai-hat-plus.html>

Hailo compiler:

<https://hailo.ai/products/hailo-software/hailo-ai-software-suite/>